aiohttp / HTTPX — Full Python Notes

1. What Are aiohttp and HTTPX?

`aiohttp` and `httpx` are Python libraries used to send HTTP requests and communicate with web servers and APIs.

They can be used to:

- Send GET, POST, PUT, PATCH, and DELETE requests.
- Fetch data from REST APIs.
- Send JSON data to APIs.
- Perform asynchronous web scraping.
- Handle multiple HTTP requests concurrently.
- Manage authentication, headers, cookies, and timeouts.

What Is Asynchronous HTTP?

In synchronous code, execution waits for a request to finish before continuing. In asynchronous code, other tasks can run while a network request is waiting for a response.

Synchronous:

```text
Request 1 → Wait → Complete → Request 2 → Wait → Complete
```

Asynchronous:

```text
Request 1 starts
Request 2 starts
Responses arrive as ready
```

Asynchronous I/O is especially useful for network-bound tasks. It does not automatically make CPU-intensive tasks faster.

2. Installation

Install both libraries:

```bash
pip install aiohttp httpx
```

Install them separately if needed:

```bash
pip install aiohttp
```

```bash
pip install httpx
```

3. aiohttp vs HTTPX

Feature	aiohttp	HTTPX
Async requests	Yes	Yes
Sync requests	Primarily an async client API	Yes
HTTP/2 support	Not a standard client feature	Optional

Async client	`ClientSession`	`AsyncClient`
Sync client	Not its main use	`Client`
Best known for	Async HTTP and web server support	Sync and async HTTP client

Which One Should You Learn?

- **HTTPX:** Best when you want to learn both synchronous and asynchronous API requests.
- **aiohttp:** Useful for asynchronous networking, high-concurrency requests, and projects that also need its asynchronous server framework.

Part 1: HTTPX Full Notes

4. Basic GET Request

A GET request retrieves data from a server.

```python
import httpx

response = httpx.get("https://httpbin.org/get")

print(response.status_code)
print(response.text)
```

Explanation

- `httpx.get()` sends a GET request.
- `response.status_code` returns the HTTP status code.
- `response.text` returns the response body as text.

Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
429	Too Many Requests
500	Internal Server Error

5. JSON Response

APIs commonly return data in JSON format.

```python
import httpx

response = httpx.get(
    "https://jsonplaceholder.typicode.com/posts/1"
)

data = response.json()

print(data)
print(data["title"])
print(data["body"])
```

`response.json()` converts a JSON response into a Python dictionary or list.

6. GET Request with Query Parameters

Query parameters allow you to request specific data from a server.

```python
import httpx

url = "https://httpbin.org/get"

params = {
    "name": "Faruk",
    "age": 25,
}

response = httpx.get(url, params=params)

print(response.url)
print(response.json())
```

HTTPX automatically adds the query parameters to the URL.

7. POST Request with JSON

A POST request is commonly used to send data to a server.

```python
import httpx

url = "https://httpbin.org/post"

payload = {
    "name": "Faruk",
    "language": "Python",
}

response = httpx.post(url, json=payload)

print(response.status_code)
print(response.json())
```

The `json=payload` argument serializes the data as JSON and sets the appropriate content type.

8. POST Request with Form Data

```python
import httpx

url = "https://httpbin.org/post"

data = {
    "username": "faruk",
    "password": "example-password",
}

response = httpx.post(url, data=data)

print(response.status_code)
print(response.json())
```

The `data=` argument sends form-encoded data.

9. Custom Headers

Headers provide additional information about an HTTP request.

```python
import httpx

headers = {
    "User-Agent": "MyPythonApp/1.0",
    "Accept": "application/json",
}

response = httpx.get(
    "https://httpbin.org/headers",
    headers=headers,
)

print(response.json())
```

Many APIs use the `Authorization` header for authentication.

```python
headers = {
    "Authorization": "Bearer YOUR_API_TOKEN"
}
```

Avoid hardcoding API tokens in source code. Use environment variables or a secret manager instead.

10. PUT, PATCH, and DELETE

```python
import httpx

url = "https://jsonplaceholder.typicode.com/posts/1"

# PUT: Replace or update a resource
response = httpx.put(
    url,
    json={
        "id": 1,
        "title": "Updated title",
        "body": "Updated body",
        "userId": 1,
    },
)

print("PUT:", response.status_code)

# PATCH: Partially update a resource
response = httpx.patch(
    url,
    json={"title": "New title"},
)

print("PATCH:", response.status_code)

# DELETE: Request deletion of a resource
response = httpx.delete(url)
```

```
print("DELETE:", response.status_code)
```

JSONPlaceholder is a testing API. Its responses do not necessarily indicate that a real database was modified.

11. Asynchronous GET Request

```python
import asyncio
import httpx


async def main():
    async with httpx.AsyncClient() as client:
        response = await client.get(
            "https://jsonplaceholder.typicode.com/posts/1"
        )

        print(response.status_code)
        print(response.json())


asyncio.run(main())
```

Important Concepts

- `async def` defines an asynchronous function.
- `AsyncClient()` creates an asynchronous HTTP client.
- `await client.get()` waits for the response while allowing the event loop to run other eligible tasks.
- `asyncio.run(main())` runs the asynchronous program.

Common mistake: Calling `client.get()` without `await` returns a coroutine rather than the HTTP response.

12. Multiple Requests Concurrently

Use `asyncio.gather()` to run multiple independent requests concurrently.

```python
import asyncio
import httpx


async def fetch_post(client, post_id):
    url = f"https://jsonplaceholder.typicode.com/posts/{post_id}"

    response = await client.get(url)
    response.raise_for_status()

    return response.json()


async def main():
    async with httpx.AsyncClient() as client:
        results = await asyncio.gather(
            fetch_post(client, 1),
            fetch_post(client, 2),
            fetch_post(client, 3),
        )
```

```
        for post in results:
            print(post["id"], post["title"])


asyncio.run(main())
```

The three requests can run concurrently. The program does not need to wait for one response before starting the next request.

Avoid sending thousands of requests simultaneously without considering API rate limits and system resources.

13. Timeout Handling

Timeouts prevent requests from waiting indefinitely.

```python
import httpx

try:
    response = httpx.get(
        "https://httpbin.org/delay/5",
        timeout=3.0,
    )

    print(response.status_code)

except httpx.TimeoutException:
    print("Request timed out")
```

Asynchronous Version

```python
import asyncio
import httpx


async def main():
    try:
        async with httpx.AsyncClient(timeout=3.0) as client:
            response = await client.get(
                "https://httpbin.org/delay/5"
            )

            print(response.status_code)

    except httpx.TimeoutException:
        print("Request timed out")


asyncio.run(main())
```

14. Error Handling

```python
import httpx

try:
    response = httpx.get(
        "https://jsonplaceholder.typicode.com/posts/1",
        timeout=10.0,
```

```
    )

    response.raise_for_status()

    data = response.json()
    print(data)

except httpx.TimeoutException:
    print("Request timed out")

except httpx.HTTPStatusError as error:
    print("HTTP error:", error.response.status_code)

except httpx.RequestError as error:
    print("Network or request error:", error)
```

`raise_for_status()` raises an `HTTPStatusError` for HTTP 4xx and 5xx responses.

Part 2: aiohttp Full Notes

15. Basic GET Request

```python
import asyncio
import aiohttp


async def main():
    async with aiohttp.ClientSession() as session:
        async with session.get(
            "https://jsonplaceholder.typicode.com/posts/1"
        ) as response:
            print(response.status)

            data = await response.json()
            print(data)


asyncio.run(main())
```

Explanation

- `ClientSession()` manages session resources and connections.
- `session.get()` creates a GET request.
- `async with` helps release the response and session resources correctly.
- `await response.json()` reads the JSON response body asynchronously.

16. Read a Text Response

```python
import asyncio
import aiohttp


async def main():
    async with aiohttp.ClientSession() as session:
        async with session.get("https://example.com") as response:
            text = await response.text()
            print(text)
```

```
asyncio.run(main())
```

- `await response.text()` reads the response body as text.
- `await response.json()` reads and parses a JSON response.

17. POST Request with JSON

```python
import asyncio
import aiohttp


async def main():
    url = "https://httpbin.org/post"

    payload = {
        "name": "Faruk",
        "language": "Python",
    }

    async with aiohttp.ClientSession() as session:
        async with session.post(url, json=payload) as response:
            print(response.status)

            data = await response.json()
            print(data)


asyncio.run(main())
```

The `json=` argument sends JSON data.

18. Query Parameters and Headers

```python
import asyncio
import aiohttp


async def main():
    url = "https://httpbin.org/get"

    params = {
        "page": 1,
        "limit": 10,
    }

    headers = {
        "Accept": "application/json",
        "User-Agent": "MyPythonApp/1.0",
    }

    async with aiohttp.ClientSession(headers=headers) as session:
        async with session.get(
            url,
            params=params,
        ) as response:
            print(response.status)
            print(await response.json())


asyncio.run(main())
```

```

## 19. Concurrent Requests with aiohttp

```python
import asyncio
import aiohttp


async def fetch_post(session, post_id):
    url = f"https://jsonplaceholder.typicode.com/posts/{post_id}"

    async with session.get(url) as response:
        response.raise_for_status()
        return await response.json()


async def main():
    async with aiohttp.ClientSession() as session:
        results = await asyncio.gather(
            fetch_post(session, 1),
            fetch_post(session, 2),
            fetch_post(session, 3),
        )

        for post in results:
            print(post["id"], post["title"])


asyncio.run(main())
```

Reusing a single `ClientSession` allows connection pooling and avoids unnecessary session creation.

20. Timeout Handling in aiohttp

```python
import asyncio
import aiohttp


async def main():
    timeout = aiohttp.ClientTimeout(total=3)

    try:
        async with aiohttp.ClientSession(
            timeout=timeout
        ) as session:
            async with session.get(
                "https://httpbin.org/delay/5"
            ) as response:
                print(response.status)

    except asyncio.TimeoutError:
        print("Request timed out")


asyncio.run(main())
```

Timeout exceptions can be handled when a request exceeds the configured limit. Check the documentation for the specific aiohttp version you use, because timeout exception classes can vary by version.
```

21. Error Handling in aiohttp

```python
import asyncio
import aiohttp


async def main():
    try:
        async with aiohttp.ClientSession() as session:
            async with session.get(
                "https://jsonplaceholder.typicode.com/posts/1"
            ) as response:
                response.raise_for_status()

                data = await response.json()
                print(data)

    except aiohttp.ClientResponseError as error:
        print("HTTP error:", error.status)

    except asyncio.TimeoutError:
        print("Request timed out")

    except aiohttp.ClientError as error:
        print("Request error:", error)


asyncio.run(main())
```

`raise_for_status()` raises an exception for unsuccessful HTTP status codes.

Part 3: Important Concepts

22. Limit Concurrent Requests

Use a semaphore to limit how many requests can run concurrently.

```python
import asyncio
import httpx


async def fetch(client, semaphore, post_id):
    url = f"https://jsonplaceholder.typicode.com/posts/{post_id}"

    async with semaphore:
        response = await client.get(url)
        response.raise_for_status()

        return response.json()


async def main():
    semaphore = asyncio.Semaphore(5)

    async with httpx.AsyncClient(timeout=10.0) as client:
        tasks = [
            fetch(client, semaphore, post_id)
            for post_id in range(1, 21)
        ]
```

```
        results = await asyncio.gather(*tasks)

        print("Fetched:", len(results))


asyncio.run(main())
```

This schedules 20 requests but allows at most five requests to execute inside the semaphore at a time.

The same technique can be used with an aiohttp `ClientSession`.

23. Connection Pooling

Connection pooling manages and reuses network connections across requests.

HTTPX

```python
import asyncio
import httpx


async def main():
    async with httpx.AsyncClient() as client:
        response1 = await client.get("https://example.com")
        response2 = await client.get("https://example.com/about")

        print(response1.status_code)
        print(response2.status_code)


asyncio.run(main())
```

aiohttp

```python
import asyncio
import aiohttp


async def main():
    async with aiohttp.ClientSession() as session:
        async with session.get("https://example.com") as response:
            print(response.status)


asyncio.run(main())
```

Reusing a client or session is generally better than creating a new one for every request.

24. When to Use Each Library

HTTPX

Use HTTPX when you need to:

- Build REST API clients.
- Make both synchronous and asynchronous requests.
- Test APIs.

- Use optional HTTP/2 support.
- Call external APIs from Django or another backend application.

aiohttp

Use aiohttp when you need to:

- Perform asynchronous networking.
- Manage many concurrent network requests.
- Use an asynchronous HTTP client and its asynchronous web server in the same ecosystem.

25. Common Mistakes

1. Forgetting `await` when making asynchronous requests.
2. Creating a new client or session for every request.
3. Not setting timeouts.
4. Ignoring HTTP errors instead of calling `raise_for_status()`.
5. Sending thousands of requests without concurrency limits.
6. Forgetting how to read the response body.
7. Ignoring API rate limits and terms of service.
8. Hardcoding API tokens in source code.

Part 4: Interview Questions

Q1. What is the difference between synchronous and asynchronous requests?

Synchronous execution waits for a response before continuing. Asynchronous execution allows the event loop to run other eligible tasks while a network operation is waiting.

Q2. What is asyncio?

`asyncio` is a Python standard library for asynchronous programming and event-loop management.

Q3. What is the difference between httpx.Client and httpx.AsyncClient?

`Client` performs synchronous requests, while `AsyncClient` performs asynchronous requests and is generally used with `await`.

Q4. What is the difference between aiohttp.ClientSession and httpx.AsyncClient?

Both manage asynchronous HTTP requests and connection resources. Their APIs, configuration options, and supported features differ.

Q5. What is asyncio.gather()?

It schedules multiple awaitables and collects their results.

Q6. What is a timeout?

A timeout is a limit on how long an operation is allowed to take before it is treated as failed.

Q7. Does asynchronous code always run faster?

No. It is especially useful for concurrent I/O-bound work. CPU-intensive tasks may require multiprocessing or another suitable parallel-computing approach.

Part 5: Practice Projects

1. **API Data Fetcher:** Fetch 10 posts from a public API and save them to a JSON file.
2. **Concurrent API Client:** Fetch data from 10 different endpoints and compare execution times.
3. **Retry and Timeout Handler:** Add a limited retry mechanism for timeouts and transient network errors.
4. **Rate-Limited Scraper:** Collect data from an authorized website using concurrency limits, timeouts, and error handling.
5. **Django API Integration:** Send a request from a Django backend to an external REST API and handle its response.

Part 6: Quick Revision Cheat Sheet

Task	HTTPX Async	aiohttp
Create client	`httpx.AsyncClient()`	`aiohttp.ClientSession()`
GET request	`await client.get(url)`	`async with session.get(url) as r:`
JSON response	`response.json()`	`await response.json()`
Text response	`response.text`	`await response.text()`
POST JSON	`await client.post(url, json=data)`	`session.post(url, json=data)`
Check HTTP status	`response.raise_for_status()`	`response.raise_for_status()`
Run async code	`asyncio.run(main())`	`asyncio.run(main())`

Part 7: Official Documentation

- HTTPX: https://www.python-httpx.org/
- HTTPX Async Guide: https://www.python-httpx.org/async/
- aiohttp: https://docs.aiohttp.org/en/stable/
- Python asyncio: https://docs.python.org/3/library/asyncio.html

Learning Recommendation

Start with HTTPX synchronous and asynchronous requests. Then practice `asyncio.gather()`, timeouts, exception handling, and concurrency limits. After that, learn aiohttp to understand the differences between the two libraries.